A Multi-Dimensional, Per-Pass Empirical Study of the LLVM Optimization Pipeline

Ph.D. Student's Activity Report

Federico Bruzzone

Computer Science Department, Università degli Studi di Milano
Supervisor: Prof. **Walter Cazzola**
Email: federico.bruzzone@unimi.it
Website: federicobruzzone.github.io

22 June 2026

Outline

Optimizing compilers and LLVM

Since the dawn of computing, compiler optimizations have long bridged high-level abstractions and efficient machine code [1–6].

Optimizing compilers [7, 8] use analysis passes to guide transformations [9–11] that enhance performance without altering observable behavior [12–14]—modern architectures (e.g., x86-64, Aarch64) make this a delicate interplay of interdependent passes [9].

LLVM [15] exemplifies this complexity, serving as the standard backend for C/C++, Rust [16], Julia [17], and via **MLIR** [18]—for *machine learning* (ML) and *deep learning* (DL) stacks [19, 20] that rely on ML/DL optimizing compilers [21–24] (e.g., NVCC[1], OpenXLA [25], TVM [26]) and automotive pipelines.[2]

Quantifying individual pass impact is therefore critical across these domains.

[1] developer.nvidia.com/cuda-llvm-compiler

[2] Tesla Full Self-Driving v14.3: stats.tessie.com/versions/2026.2.9.6

Callout boxes and dingbats

Definition

A clean callout with a colored left rule.

Warning

A red box for caveats.

Tip

A green box for tips and examples.

Inline boxes: **info**, **warn**, **note**.

Colored dingbats, ported from SCRIBE:

- ✔ done
- ✘ failed
- ☞ note

[jd] *Another comment, in blue, from John Doe.*

Mathematics

The factorial, set with the Libertine math defaults:

$$fact(n) = \begin{cases} 1 & \text{if } n = 0 \\ n \cdot fact(n-1) & \text{if } n > 0 \end{cases}$$

Code listings

Monospace is set in Inconsolata:

```
int fact(int n) {
    if (n == 0) return 1;
    return n * fact(n - 1);
}
```

Summary

- One theme file: `beamerthemescribe.sty`
- Same fonts, colors, and ornament as `scribe.cls`
- Toggle comments with `\scribeshowcommentsfalse`

Thank you!

federicobruzzone.github.io

References I

[1] A. P. Ershov. On Programming of Arithmetic Operations. *Communications of the ACM*, 1(8):3–6, 1958.

[2] J. Heller. Sequencing aspects of multiprogramming. *J. ACM*, 8(3):426–439, 1961.

[3] W. M. McKeeman. Peephole optimization. *Commun. ACM*, 8(7):443–444, 1965.

[4] C. W. Gear. High speed compilation of efficient object code. *Commun. ACM*, 8(8):483–488, 1965.

[5] Edward S. Lowry and C. W. Medlock. Object Code Optimization. *Communications of the ACM*, 12(1):13–22, January 1969.

[6] Vincent A. Busam and Donald E. Englund. Optimization of expressions in fortran. *Communications of the ACM*, 12(12):666–674, 1969.

[7] Frances E. Allen. Program optimization. In Mark Halpern and Christopher Shaw, editors, *Annual Review of Automatic Programming*, volume 5, pages 239–307. Pergamon Press, 1966.

[8] William Allan Wulf. The design of an optimizing compiler. 1973.

[9] Keith D. Cooper and Linda Torczon. *Engineering a Compiler*. Morgan Kaufmann, November 2022.

[10] Ken Kennedy and John R. Allen. *Optimizing Compilers for Modern Architectures: A Dependence-Based Approach*. Morgan Kaufmann Publishers Inc., October 2001.

[11] R. Paige. Future directions in program transformations. *SIGPLAN Not.*, 32(1):94–98, 1997.

[12] David F. Bacon, Susan L. Graham, and Oliver J. Sharp. Compiler Transformations for High-Performance Computing. *ACM Computing Surveys*, 26(4):345–420, December 1994.

References II

[13] Mike Dodds, Mark Batty, and Alexey Gotsman. Compositional verification of compiler optimisations on relaxed memory. In *Programming Languages and Systems*, pages 1027–1055. Springer, 2018.

[14] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, Boston, MA, USA, second edition, 2006.

[15] Chris Lattner and Vikram Adve. LLVM: A Compilation Framework for Lifelong Program Analysis and Transformation. In Michael D. Smith, editor, *Proceedings of the 2nd International Symposium on Code Generation and Optimization (CGO'04)*, pages 75–86, San José, CA, USA, March 2004. IEEE.

[16] Nicholas D. Matsakis and Felix S. Klock. The Rust Language. *ACM SIGAda Letters*, 34(3):103–104, October 2014.

[17] Jeff Bezanson, Stefan Karpinski, Viral B Shah, and Alan Edelman. Julia: A fast dynamic language for technical computing. *arXiv:1209.5145*, 2012.

[18] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. Mlir: Scaling compiler infrastructure for domain specific computation. In *CGO '21*, pages 2–14. IEEE, 2021.

[19] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Adv. Neural Inf. Process. Syst.*, 32, 2019.

[20] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. TensorFlow: a system for Large-Scale machine learning. In *OSDI '16*, pages 265–283, 2016.

References III

[21] Mingzhen Li, Yi Liu, Xiaoyan Liu, Qingxiao Sun, Xin You, Hailong Yang, Zhongzhi Luan, Lin Gan, Guangwen Yang, and Depei Qian. The deep learning compiler: A comprehensive survey. *IEEE Trans. Parallel Distrib. Syst.*, 32(3):708–727, 2020.

[22] Yu Xing, Jian Weng, Yushun Wang, Lingzhi Sui, Yi Shan, and Yu Wang. An in-depth comparison of compilers for deep neural networks on hardware. In *ICESS '19*, pages 1–8, 2019.

[23] Ruizhe Zhao, Shuanglong Liu, Ho-Cheung Ng, Erwei Wang, James J. Davis, Xinyu Niu, Xiwei Wang, Huifeng Shi, George A. Constantinides, Peter Y. K. Cheung, and Wayne Luk. Hardware compilation of deep neural networks: An overview. In *ASAP '18*, pages 1–8, 2018.

[24] Hsin-I Cindy Liu, Marius Brehler, Mahesh Ravishankar, Nicolas Vasilache, Ben Vanik, and Stella Laurenzo. Tinyiree: An ml execution environment for embedded systems from compilation to deployment. *IEEE Micro*, 42(5):9–16, 2022.

[25] Amit Sabne. Xla: Compiling machine learning for peak performance, 2020.

[26] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Haichen Shen, Eddie Q Yan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. Tvm: end-to-end optimization stack for deep learning. *arXiv:1802.04799*, 2018.